ENP RACING TEAM

VERSION 1.1 · 11 Oct 2026

aero_auto 1.1 User Manual

Dark edition - run the GUI, schedule campaigns, read the reports.

The practical guide: what you need, how to install it, how to run every tool from the GUI or the command line, and what lands in the report folder. Engineers looking for the "why" behind the code should open the technical guide.

Six tabs: Configure / Airfoils / Campaigns / Results / Graphs / Optimizer
Settings remembered between sessions
Menus: File / View / Help

Contents

1	Overview
2	Requirements and dependencies
3	Install
4	Run
5	The GUI, tab by tab (run_ui.sh / aero_ui.exe)
6	File locations at a glance
7	Environment overrides
8	What a campaign writes
9	The report folder
10	Regulatory reference
11	Packaging and platform notes
12	Changelog highlights

1 Overview

Module	Purpose
aero_auto.py	core engine - rules, geometry, JavaFoil, planner
aero_targets.py	CL*A / CD*A target report
aero_report.py	ranked tables, workbook, graphs, --resim
flowfield.py	schematic potential-flow model for field images
aero_ui.py	the GUI shell
render_field.py	headless field PNG renderer
launcher.py	one entry point for every tool

2 Requirements and dependencies

Python packages (requirements.txt): numpy, pandas, matplotlib, openpyxl, Pillow (icon build); pyinstaller is only needed for standalone builds.

System tool	Needed for	Notes
Python 3.9+ with Tkinter	everything; the GUI uses Tk	python.org installer bundles Tk on Windows
Java 8+ (java)	simulation	javafoil.jar required in JAVA_DIR
faketime	Linux only	skipped automatically on Windows (2.3 of the guide)
javafoil.jar + mhclasses.jar	simulation	~/ MH-AeroTools / JavaFoil / - verified present
airfoil .dat library	geometry and polars	override with AERO_AIRFOILS

3 Install

```
python3 -m pip install -r requirements.txt

# optional standalone (no Python on the target machine):
python3 create_standalone.sh     # linux/macos -> dist/aero_ui/
create_standalone.bat            # windows     -> dist\aero_ui\aero_ui.exe
```

4 Run

GUI

```
./run_ui.sh            # linux/macos
run_ui.bat             # windows
# or
python3 launcher.py ui
```

Headless tools

```
python3 launcher.py campaign --role fw --flow ground --height 300 --hours 0.5
python3 launcher.py report   --results ~/aero_campaign/results.csv --top 3
python3 launcher.py report   --results .../ground/h300pct/results.csv --resim
python3 launcher.py targets  --results ~/aero_campaign/results.csv --role fw
python3 launcher.py field    --main s1223 --flap e423,30,-6,100,5 --out f.png
                              --ground-height 300
```

5 The GUI, tab by tab (run_ui.sh / aero_ui.exe)

The window has a menu bar (File / View / Appearance / Help), seven tabs - Configure, Airfoils, Campaigns, Results, Graphs, Optimizer and Themes & Settings - and a status bar at the bottom. Launch it with **run_ui.sh** on Linux, **run_ui.bat** on Windows, or simply double-click **aero_ui.exe** in the standalone distribution (dist/aero_ui/). The same engine runs behind the window, so a config you have just sanity-checked can be simulated, campaigned or optimised without ever leaving the app. Every JavaFoil operation runs in a worker thread - the window never freezes while a polar is computed.

Every control carries a **hover description**. A single shared popup controller shows it after a short delay using a high-contrast colour pair, and always dismisses it on click, Esc, mouse-leave or tab change, so a description can never linger over the window. Descriptions can be turned off or tuned under Themes & Settings.

GLOBAL CHROME

Menus and status bar

Item	What it does
File > Open output folder / Open current results file	jump straight to the campaign output or the loaded results.csv
File > Write report (Ctrl+R)	builds the full report folder for the current results file
File > Reload airfoils / Quit (Ctrl+Q)	re-scan the airfoil store; close the app (settings saved first)
View > Refresh (F5) / Reset results filter / Clear campaign log	redraw the Results table + Graphs list; reset the table view; clear the log
View > Jump to ... tab	switch tabs from the keyboard (also Alt+1 … Alt+7, Ctrl+Tab)
Appearance > Dark / Light / High contrast	switch the colour theme instantly, or open the Themes & Settings tab
Help > About / Open airfoil store / JavaFoil / project folder	the folders you care about, one click away
status bar	live readout: airfoil count, OUT_DIR, JavaFoil jar health, last reported results file

Window layout and every tab's settings are remembered between sessions - saved to **~/.aero_auto/ui_state.json** on close and restored on start (the app never forgets your pool selections, ground heights or results path).

5.0 Run library

The app manages its own results library: every single run, batch, campaign and optimizer session is written into a fresh dated folder **<OUT_DIR>/library/<stamp>_<kind>/** containing a **results.csv**, an optional **graphs/** and **Best/** report, and a **meta.json** manifest (date, kind, main airfoil, run count, best score). The Run Library tree on the Results tab lists them - click a row to load it - with **Import existing** (copy an old results.csv in), **Export run** (zip the folder) and **Open best sheet**. Existing pre-library data (**<OUT_DIR>/results.csv** and **<OUT_DIR>/ground/*/**) is auto-imported into the library on first launch.

5.1 Configure tab

Control	What it does
Main element	the airfoil that defines the wing; the combo lists every profile found in your store and .dat folder
Role: front / rear / custom	sets the span used by CL*A and the force model (fw 1400 / rw 950 / custom span, editable) and the FSUK/FSG rule set that is checked
Main chord (mm)	physical chord in millimetres; enables the all-mm geometry columns and the force conversion
Speed (m/s)	freestream speed for q = 0.5 * rho * V^2 in the force model
Reynolds number	run Reynolds number sent to JavaFoil (default 570 000)
Alpha start / end / step (deg)	polar sweep angles; the default -20..+5 step 1 covers the stall search

Control	What it does
Flap rows F1..F3	per slot: airfoil combo + S% scale of main chord, A deg mount angle, X% position, Y% position; tick the checkbutton to enable the row
derived values readout	live label of the running config: Reynolds, dynamic pressure q, and the CL*A / CD*A targets for the selected role
enforce FSUK/FSG size limits	master switch for the regulatory checks (stack height/length, ground clearance)
Ground effect / Ground height (% chord)	wing height above the ground for ground-effect runs; off or empty = unbounded flow. When on it must be a positive number (the ground is y = 0), enforced centrally for every entry point
Sanity check geometry	instant PASS/FAIL against the size limits - prints the exact mm value vs limit and the per-pair gap/overlap in %, then redraws the preview (green PASS / red FAIL, ground line drawn when a ground height is set)
Simulate this wing now (JavaFoil)	runs the real JavaFoil analysis in a worker thread, writes the row into a new dated run folder in the library and logs Run_ID, Status, Cl@0, Cd@0, max L/D, stall angle and Downforce_N
Run in Terminal / Batch: Multiple Main/Flaps	run the same simulation in a separate konsole window, or sweep several main/flap combinations in one go
status log + geometry preview	right-hand pane: live log of every action and an embedded matplotlib drawing of the element stack scaled to mm (skipped silently if matplotlib is unavailable)

5.2 Airfoils tab

Control	What it does
store location label	shows the internal airfoil store folder <APP_DATA_DIR>/airfoils (default ~/.aero_auto/airfoils), auto-created on first use
Filter store	live filter box - type to narrow the list; the count label shows matched / total
Import .dat...	copies any airfoil .dat file into the internal store under the name you type; the profile is then available in every combo - you no longer have to remember which folder holds a profile
Remove selected	deletes the selected stored profile from the internal store (only affects the store, never your original .dat files)
Open store folder	opens the store folder in the file manager when you want to inspect or manually drop files in
Refresh	re-scans the store (and AIRFOIL_DIR) in case files changed on disk
store list	shows every profile currently inside the internal store with its full path; select a row to remove it

5.3 Campaigns tab

The left pane is the setup, the right pane is a live monitor - you can watch a campaign progress, skip a stuck scenario or stop everything, all while the rows stream into the output file.

Control	What it does
Hours	campaign time budget for the planner (0.1 works as a smoke test)
Workers	number of parallel JavaFoil worker directories
Out folder + Browse / Open	where the campaign writes (default OUT_DIR); ground-effect runs get their own ground/<height>pct/ subfolders here
Flow: Unbounded / Ground effect	free stream, or ground effect at one or more ride heights entered as a comma list (e.g. 50,100,300 % chord)
Role: Rear / Front / Custom	which FSUK/FSG rule set and CL*A span to use
Main elements / Flap pool	multi-select lists of the airfoils the planner may explore, with All / None / Inv helpers
Resume previous results.csv	on - the planner detects the output folder's existing rows, keeps consistent row counts and continues RUN_COUNTER instead of overwriting history; off - starts fresh

Control	What it does
Enforce FSUK/FSG size limits	regulatory gate on every generated candidate
Self-check before each scenario	quick JavaFoil + faketime validation per scenario; failures mark the scenario skipped instead of poisoning the campaign
Write report / Open results folder when finished	post-campaign actions
Start campaign / Stop all / Skip scenario	run control; stop and skip take effect between scenarios
Self-check only	validates JavaFoil, faketime and every scenario without simulating
Write report now / Copy plan	build the report without leaving the app; copy the full campaign plan (per-scenario budgets, mains, flap pool) to the clipboard
live stats label	elapsed time, simulations finished, current runs/second
Scenarios tree	one row per scenario with live Status and Runs; double-click a row to open that scenario's folder
Current best (live)	the best config found so far, updated as runs finish
Campaign log	streamed log with Clear / Save... / Copy and an auto-scroll toggle; capped so long campaigns stay responsive

5.4 Results tab

Control	What it does
Run Library tree	one row per dated run/campaign in <OUT_DIR>/library (date, kind, main, best score, run count); click a row to load it
Refresh / Import existing results...	re-scan the library, or copy any existing results.csv (with its graphs/ and Best/) into it as a dated import run
Export run... / Open best sheet	zip the selected run folder, or open its Best/ report folder
results.csv path entry + Load	which result file to inspect (default: the newest library run)
Browse...	pick any results.csv on disk (e.g. a legacy ground folder)
Show targets	runs the CL*A / CD*A target report for the currently selected role and prints the ranked top-10 into the log
live filter box	keep only rows whose text matches in any column (case-insensitive, instant); empty = show all
sortable headers	click a column header to sort, click again to reverse; numeric columns sort numerically, not alphabetically
stats footer	shows everything the current view matches: total simulations, summed sim time, first-to-last date
results tree	one row per config: Run_ID, Date, Elements, Main, Cl_a0, Cd_a0, Max_LD, Downforce_N, Duration_s, Tier, Score, Status. Run_ID/Date/Duration_s tell you how many simulations exist, when a run happened and how long it took (your total time worked); up to 1000 rows rendered from the current view
Graphs for selected run	renders polar, geometry and field PNGs for the highlighted row straight into <results-dir>/graphs and opens the Graphs tab (clicking any row does this automatically, rebuilding only what is missing)
detail pane	clicking a row shows the full picture: date, duration, alpha interval + RE, flap stack, flow type, polar metrics, downforce/drag forces and the mm geometry
CSV Data Viewer	raw CSV text of the loaded file with Refresh / Save for manual inspection and small edits

5.5 Graphs tab

The Graphs tab is the image side of the app. It discovers every results file the app knows about - the dated library runs plus any legacy <OUT_DIR> / ground folders - and any generated graph images, previewing

them right inside the window.

Control	What it does
results file tree	one row per results file (library run or legacy folder) with its modified date and PNG count; click to load it
Write report	builds the full report folder for the selected results file (also used to update the images after a new campaign)
Refresh	re-scans the selected results folder for polar/geometry/field PNGs
Open in viewer	opens the current image in your system image viewer
image list	every <results-dir>/graphs/*.png found for the chosen results file
preview	click an image name to see it scaled inside the app (needs Pillow; without it the list still works, just no inline preview)

5.6 Optimizer tab

A hill-climbing optimizer over flap geometry (and main choice) that repeatedly simulates candidates and keeps the best. The winning configuration is pushed back into the Configure tab with **Apply Best** and its final run is appended to the results file.

Control	What it does
Base Wing Configuration	manual entry like Configure: main element, results file, flap rows F1..F3 (S/A/X/Y + enable), chord / speed / Re / alpha, ground effect + height
Optimization Objectives	weighted objectives: Lift (Cl), Efficiency (Cl/Cd), Stability (min Cl variation), Stack Height, Stack Length, Target Gap, Downforce, Efficiency @ target Cl
Target Values	target Cl, target gap %, max height (mm), max length (mm) used by the corresponding objectives
Optimization Parameters	iterations, step size, perturb scale, and an enforce-regulations switch (height / length / gap constraints)
Progress bar + status	current iteration out of the total; live status text
Start Optimization / Stop / Apply Best	run control; stop lands between iterations; Apply Best sends the winning configuration back to Configure

5.7 Themes & Settings tab

Controls the look of the app and the hover descriptions. Every change is written straight to **ui_state.json**, so it survives a restart. The three palettes live in one **THEMES** dict in aero_ui.py; the dark one is the same palette the PDFs use (docs/theme.py). Switching a theme rebuilds the chrome and tabs under the new colours in place.

Control	What it does
Theme: Dark / Light / High contrast	three full palettes with colour swatches; picking one re-themes the whole window instantly
Show hover descriptions	turn the hover help on or off (off by default only if you disable it)
Delay before showing (ms) / Wrap width (px)	how long the mouse must rest before a description appears, and how long its lines may get
Preview description	pops a sample description built with the current settings
Navigation cheatsheet	Alt+1 … Alt+7, Ctrl+Tab, Ctrl+R, F5, Esc, Ctrl+Q

> **Note.** Descriptions use a high-contrast pair (light card/dark text on the dark themes, dark card/light text on the light theme) and only one can be on screen at a time; it is dismissed on click, Esc, mouse-leave or tab change.

6 File locations at a glance

Where airfoils are read from and every kind of output lands. All paths can be moved with the environment overrides in chapter 7.

Artifact	Location
airfoil .dat files	default AIRFOIL_DIR "/mnt/data/FSEA/AIRFOILS JAVAFOIL/AIRFOILS" - put one .dat per profile in this folder (top-level files only, no sub-folders). The GUI combos are built from these filenames. Set AERO_AIRFOILS to point elsewhere
run library root	<OUT_DIR>/library/ (default ~/aero_campaign/library/) - one dated folder <stamp>_<kind>[_label]/ per run, each with meta.json
single simulations / GUI campaigns	<run-dir>/results.csv inside the library run folder; the raw engine aero_auto.py, run by hand, still writes <OUT_DIR>/results.csv
ground-effect campaigns (CLI/GUI)	<run-dir>/ground/<height>pct/results.csv - one file per ride height, inside the campaign run folder
ranked tables	<results-dir>/Best/configs.csv, Best/overview.txt, Best/overview.xlsx (next to whichever results.csv you reported)
readable CSV + legend	<results-dir>/results_human.csv and <results-dir>/columns_legend.csv
polar and geometry graphs	<results-dir>/graphs/polar_<Run>.png and geometry_<Run>.png
velocity maps (field sims)	<results-dir>/graphs/field_<Run>.png - schematic potential-flow velocity field for that config; the render_field.py CLI produces a single picture from any configuration
target report	targets_report.csv next to the results file you pointed the tool at

Note. The polar graphs are the authoritative data. field_*.png images are a schematic potential-flow visualisation whose circulation comes from each config's own JavaFoil Cl - not a second force solver.

7 Environment overrides

Variable	Effect
AERO_JAVA_DIR	folder that contains javafoil.jar
AERO_AIRFOILS	folder with the .dat profiles
AERO_OUT	campaign output folder (default ~/aero_campaign)
AERO_ICON	explicit GUI icon path

8 What a campaign writes

~/aero_campaign/results.csv holds unbounded runs; ground-effect runs land in ~/aero_campaign/ground//results.csv. Rows carry the configuration, the polar metrics, per-pair geometry, forces and quality:

```
Run_ID Phase Main F1_Airfoil F1_Scale F1_Angle F1_X F1_Y F2_* F3_* \
Cl_peak Cd_peak Stall_Angle Max_LD Cl_a0 Cd_a0 Cm_a0 Cl_am2 Cl_ap2 \
Downforce_N Drag_N Tier Score Status  + per-pair gap/overlap/step + mm geometry
```

9 The report folder

File	Content
Best/configs.csv	the top-N, round-robin across stack sizes
Best/overview.txt	human summary incl. spans, q and force model

File	Content
Best/overview.xlsx	Top configs / All rows / Legend sheets
results_human.csv	every valid row, readable column names
columns_legend.csv	column - unit - description for everything
graphs/*.png	polar, geometry and field images per config

Note. Polar graphs are the authoritative data. field_*.png images are a schematic potential-flow visualisation, not a second force solver.

10 Regulatory reference (FSUK 2026 / FSG v1.1)

Rule	Limit
Min ground clearance	35 mm
Front wing max stack height	250 mm
Front wing max stack length	600 mm
Rear wing max total height	1100 mm
Rear wing max stack length	250 mm
Front wing span	1400 mm
Rear wing span	950 mm
Targets CL*A / CD*A roof	fw 1.75 / 0.45 - rw/beam 1.85 / 0.95 (tol ±0.4)

11 Packaging and platform notes

Four distributions come from the same modules: the Linux tar.gz, the Windows zip, the installed desktop app (~/Applications/aero_auto, registered via the built-in .desktop entry), and the standalone aero_ui built with PyInstaller (the .exe under wine). The standalone folder bundles the app, assets and both docs with --add-data and --add-binary, so a shipped copy carries its own documentation. Versioned archives are produced by build_dist.py; the PyInstaller folder by create_standalone.sh / create_standalone.bat.

12 Changelog highlights

1.0

- regulatory engine + GUI verdict, geometry_ok gate before JavaFoil
- ground-effect campaigns and per-height result files
- force model (Downforce_N / Drag_N) and all-mm geometry columns
- target report against CL*A / CD*A roofs
- calibrated potential-flow model (replaces the abandoned linear solver)
- full report pipeline: workbook, legend, duplicate-safe headers, --resim
- threaded Tk GUI with a queue-based thread bridge and icon support
- cross-platform packaging: launcher, field CLI, desktop app, wine-built exe

1.1 (this build)

- Campaigns tab redesigned into setup + live monitor: per-scenario tree with live status/runs, live best config, stats (elapsed / finished / runs-per-second), Skip scenario / Stop all, Self-check only, Copy plan to clipboard, resume-prevents-overwrite

shared run_plan engine for CLI and GUI campaigns: per-scenario time budgets, a strict deadline, scenario progress events

Optimizer tab: objectives, target values, constraints, live progress, Apply Best

Results tab: sortable columns, live filter, stats footer, detail pane, CSV data viewer

menu bar (File / View / Help with Ctrl+R, F5, Ctrl+Q), status bar, settings persistence to ~/.aero_auto/ui_state.json

legacy results.csv auto-upgrade with backup (old 31/47-column rows are recovered instead of dropped)

optimizer scoring re-wired through the UI queue (the old opt bridge was never scheduled)

docs regenerated for 1.1 in the dark edition

1.1 (library + dark UI)

dark theme across the whole app (matches the docs palette), header band and themed ttk widgets; dark geometry preview

app-managed run library: every run becomes a dated folder under /library with a meta.json manifest; existing results.csv / ground data auto-imported on first launch

Run Library tree on the Results tab with Import existing / Export run (zip) / Open best sheet

click a results row to auto-generate its graphs (only when missing) and jump to the Graphs tab

campaigns auto-build the full report (Best/ + graphs) into each scenario folder when finished

central ground-height guard: heights must be positive, enforced for every entry point including aero_targets --height

1.1 (themes + descriptions)

Themes & Settings tab: Dark / Light / High contrast palettes with live switching (the window rebuilds under the new colours) plus hover-description and navigation settings, all persisted to ui_state.json

hover descriptions rebuilt as one shared, delayed, high-contrast popup that is dismissed on click / Esc / leave / tab change, so none keeps floating

Appearance menu and Alt+1 … Alt+7 / Ctrl+Tab tab navigation